

Stage 4: Exploratory Data Analysis (EDA) & Predictive Modeling

Objective: In this stage, The data will be analysed to extract business insights and answer the core questions defined in the "Ask" phase. I will also use Machine Learning to measure how operational issues impact customer satisfaction. The goal is to translate raw data into clear findings before visualizing them in Stage 5.

Note: Make sure the `animesh_env` conda environment is activated for this notebook to run properly. Check the `readme.md` for setup instructions.

Load data from MySQL database

1. Establish the Connection using utils

```
In [1]: # 1. Import local security utility and standard libraries
import pandas as pd
import numpy as np
from utils import get_db_connection

# 2. Establish secure connection
# This will trigger the hidden password prompt
engine = get_db_connection()
```

INFO: Successfully connected to the MySQL database.

2. Read Data into a DataFrame

```
In [2]: # 3. Fetch the Master Analytical View
# We pull the 'master_analytical_view' created in the cleaning stage
query = "SELECT * FROM master_analytical_view"
df = pd.read_sql(query, con=engine)

print(f"--- Data Pipeline Verified ---")
print(f"Total records fetched for analysis: {len(df)}")
df.head()
df.info()
```

```

--- Data Pipeline Verified ---
Total records fetched for analysis: 110167
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 110167 entries, 0 to 110166
Data columns (total 43 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   order_id                                  110167 non-null object
1   customer_id                              110167 non-null object
2   order_status                             110167 non-null object
3   order_purchase_timestamp                 110167 non-null datetime64[ns]
4   order_approved_at                       110167 non-null datetime64[ns]
5   order_delivered_carrier_date            110167 non-null datetime64[ns]
6   order_delivered_customer_date           110167 non-null datetime64[ns]
7   order_estimated_delivery_date           110167 non-null datetime64[ns]
8   purchase_year                           110167 non-null int64
9   purchase_month                          110167 non-null int64
10  purchase_day_of_week                    110167 non-null object
11  purchase_hour                           110167 non-null int64
12  is_weekend                             110167 non-null int64
13  delivery_time_days                      110167 non-null int64
14  delivery_delay_days                     110167 non-null int64
15  is_late_delivery                        110167 non-null int64
16  customer_unique_id                     110167 non-null object
17  customer_zip_code_prefix                110167 non-null object
18  customer_city                           110167 non-null object
19  customer_state                          110167 non-null object
20  total_payment_value                     110167 non-null float64
21  payment_installments                    110167 non-null float64
22  payment_type                            110167 non-null object
23  order_item_id                           110167 non-null int64
24  product_id                              110167 non-null object
25  seller_id                               110167 non-null object
26  shipping_limit_date                     110167 non-null datetime64[ns]
27  price                                   110167 non-null float64
28  freight_value                           110167 non-null float64
29  total_item_value                        110167 non-null float64
30  freight_ratio                           110167 non-null float64
31  product_category_name                   110167 non-null object
32  product_name_lenght                     110167 non-null float64
33  product_description_lenght              110167 non-null float64
34  product_photos_qty                      110167 non-null float64
35  product_weight_g                        110167 non-null float64
36  product_length_cm                       110167 non-null float64
37  product_height_cm                       110167 non-null float64
38  product_width_cm                        110167 non-null float64
39  product_volume_cm3                      110167 non-null float64
40  review_score                            109338 non-null float64
41  has_text_review                         109338 non-null float64
42  review_response_time_days               109338 non-null float64
dtypes: datetime64[ns](6), float64(17), int64(8), object(12)
memory usage: 36.1+ MB

```

Analyzing the Database to Answer "Ask" Stage Questions

Topic 1: Sales & Revenue Trends (The Bottom Line)

Question 1.1: Month-over-Month Growth

- **Stakeholder Question:** What is the monthly revenue, and when do sales peak?
- **Analyst Question:** What is the total revenue generated month-over-month, and are there distinct seasonal spikes (e.g., holiday seasons)?
- **What to Look For:** We want to spot seasonal trends, like spikes during Black Friday in November. If there is a huge drop in a random month, it usually means there is missing data for that time period, not necessarily a drop in real sales.

```
In [3]: print("--- 1.1 Month-over-Month Revenue Growth ---\n")

# Keep the full unique orders for logistics & cancellation analysis
# Filters out 'canceled', 'unavailable', or other incomplete states
successful_orders = df[df['order_status'] == 'delivered'].drop_duplicates(subset=['order_id'])
# Calculate MoM Growth using only successful orders
monthly_revenue = successful_orders.groupby(['purchase_year', 'purchase_month'])['total_payment_value'].sum()
monthly_revenue['year_month'] = monthly_revenue['purchase_year'].astype(str) + '-' + monthly_revenue['purchase_month'].astype(str)
monthly_revenue['mom_growth_pct'] = monthly_revenue['total_payment_value'].pct_change() * 100 # Just ;

print(monthly_revenue[['year_month', 'total_payment_value', 'mom_growth_pct']])
```

--- 1.1 Month-over-Month Revenue Growth ---

	year_month	total_payment_value	mom_growth_pct
0	2016-10	46566.71	NaN
1	2016-12	19.62	-99.957867
2	2017-01	125584.67	639984.964322
3	2017-02	271298.65	116.028477
4	2017-03	414232.07	52.684899
5	2017-04	390649.57	-5.693065
6	2017-05	566872.73	45.110292
7	2017-06	490061.61	-13.549976
8	2017-07	566291.94	15.555254
9	2017-08	645898.24	14.057467
10	2017-09	701170.01	8.557350
11	2017-10	751140.27	7.126697
12	2017-11	1153393.22	53.552308
13	2017-12	843199.17	-26.894041
14	2018-01	1078606.86	27.918397
15	2018-02	966510.88	-10.392663
16	2018-03	1120678.00	15.950893
17	2018-04	1132933.95	1.093619
18	2018-05	1128836.69	-0.361650
19	2018-06	1011561.35	-10.389044
20	2018-07	1026991.01	1.525331
21	2018-08	984679.95	-4.119906

Question 1.2: Peak Traffic

- **Stakeholder Question:** Which days and hours get the most orders?
- **Analyst Question:** Which day of the week (purchase_day_of_week) and hour of the day (purchase_hour) see the highest volume of successful orders?
- **What to Look For:** We are looking for the busiest times. E-commerce usually peaks on weekday afternoons. Knowing the exact 3-hour window with the most traffic tells the marketing team exactly when to send promotional emails or launch flash sales.

```
In [4]: print("--- 1.2 Peak Traffic (Orders by Day & Hour) ---\n")

# Prepare Data
heatmap_data = successful_orders.groupby(['purchase_day_of_week', 'purchase_hour'])['order_id'].count().reset_index()
# Reorder days logically
days_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
heatmap_data = heatmap_data.reindex(days_order)

print("Heatmap matrix ready. \n\nData:\n", heatmap_data)
print("\nDimensions:", heatmap_data.shape)
```

--- 1.2 Peak Traffic (Orders by Day & Hour) ---

Heatmap matrix ready.

Data:

	0	1	2	3	4	5	6	7	8	9	...	14	\
purchase_hour													
purchase_day_of_week											...		
Monday	315	132	63	34	21	22	64	156	471	752	...	1063	
Tuesday	300	151	79	28	28	24	69	216	510	845	...	1095	
Wednesday	385	171	81	31	33	24	90	208	497	798	...	1027	
Thursday	345	165	74	38	30	28	82	211	495	736	...	950	
Friday	412	210	66	46	39	35	91	203	487	754	...	936	
Saturday	308	167	65	41	25	23	51	102	245	421	...	649	
Sunday	256	137	68	41	27	26	30	102	202	341	...	660	

	15	16	17	18	19	20	21	22	23
purchase_hour									
purchase_day_of_week									
Monday	1047	1060	954	891	920	998	1083	962	697
Tuesday	1005	1059	942	852	894	946	995	939	676
Wednesday	952	1007	942	818	822	881	927	855	599
Thursday	900	1042	881	759	798	816	826	835	532
Friday	948	943	791	697	757	714	700	675	496
Saturday	699	675	674	693	729	707	645	535	422
Sunday	696	688	775	875	881	944	860	857	591

[7 rows x 24 columns]

Dimensions: (7, 24)

Question 1.3: Average Order Value (AOV)

- **Stakeholder Question:** What is the Average Order Value (AOV) per year?
- **Analyst Question:** What is the average amount a customer spends per transaction, and how does this change over the years?
- **What to Look For:** A healthy business should see its AOV stay steady or grow. If the total number of orders goes up but the AOV drops a lot, it means people are buying more often but choosing cheaper items.

```
In [5]: print("--- 1.3 Average Order Value (AOV) ---")

# Create orders_unique to ensure we don't double-count orders with multiple items
orders_unique = df.drop_duplicates(subset=['order_id']).copy()

# Calculating Overall Historical AOV
overall_aov = orders_unique['total_payment_value'].mean()
print(f"\nOverall Historical AOV: R$ {overall_aov:.2f}")

# Calculating AOV by Year
aov_by_year = orders_unique.groupby('purchase_year')['total_payment_value'].mean().reset_index()
aov_by_year.rename(columns={'total_payment_value': 'average_order_value'}, inplace=True) # just for analysis
aov_by_year['average_order_value'] = aov_by_year['average_order_value'].round(2) # just for analysis
print("\nAOV by Year:\n", aov_by_year)
```

--- 1.3 Average Order Value (AOV) ---

Overall Historical AOV: R\$ 159.85

AOV by Year:

	purchase_year	average_order_value
0	2016	174.50
1	2017	159.41
2	2018	160.14

Topic 2: Logistics & Operational Efficiency

Question 2.1: Delivery Baselines

- **Stakeholder Question:** What is the average delivery time, and which states have the slowest deliveries?

- **Analyst Question:** What is the average delivery_time_days across the platform, and which specific states (customer_state) suffer from the slowest average deliveries?
- **What to Look For:** We expect to see states in the North or Northeast of Brazil (like RR, AP, AM) taking the longest time. This is because they are geographically far away from the main industrial hubs in the South.

```
In [6]: print("--- 2.1 Average Delivery Time by State (Top 10 Slowest): ---\n")

# Use orders_unique to prevent multi-item orders from skewing the average
state_delivery_times = orders_unique.groupby('customer_state')['delivery_time_days'].mean().sort_values(ascending=True)

print(state_delivery_times.head(10))

--- 2.1 Average Delivery Time by State (Top 10 Slowest): ---

customer_state
RR      28.975610
AP      26.731343
AM      25.986207
AL      24.040302
PA      23.316068
MA      21.117155
SE      21.029851
CE      20.817826
AC      20.637500
PB      19.953578
Name: delivery_time_days, dtype: float64
```

Question 2.2: The Cost of Delay

- **Stakeholder Question:** How many orders are delivered late?
- **Analyst Question:** What percentage of total orders are flagged as is_late_delivery?
- **What to Look For:** We need to find the overall percentage of packages that arrive late. Anything around 7-8% or higher is a major operational issue that we need to highlight for the stakeholders.

```
In [7]: print("--- 2.2 System-Wide Late Delivery: ---\n")

# Filter only delivered orders to compare estimated vs actual delivery dates
delivered_orders = orders_unique[orders_unique['order_status'] == 'delivered'].copy()

# Ensure datetime format
delivered_orders['order_estimated_delivery_date'] = pd.to_datetime(delivered_orders['order_estimated_delivery_date'])
delivered_orders['order_delivered_customer_date'] = pd.to_datetime(delivered_orders['order_delivered_customer_date'])

# Create a boolean column for late deliveries
delivered_orders['is_late'] = delivered_orders['order_delivered_customer_date'] > delivered_orders['order_estimated_delivery_date']

# Count occurrences
delivery_performance = delivered_orders['is_late'].value_counts().rename(index={False: 'On Time / Early', True: 'Late'})

# Calculate Late percentage on unique orders only
late_percentage = delivered_orders['is_late'].mean() * 100 # just for analysis

print(delivery_performance)
print(f"\nSystem-Wide Late Delivery Rate: {late_percentage:.2f}%")

--- 2.2 System-Wide Late Delivery: ---

is_late
On Time / Early    88622
Late                7825
Name: count, dtype: int64

System-Wide Late Delivery Rate: 8.11%
```

Question 2.3: Freight Impact

- **Stakeholder Question:** Does high shipping cost cause order cancellations?
- **Analyst Question:** How does the freight_ratio (the cost of shipping relative to the item price) affect purchasing behavior? Do higher freight ratios lead to more canceled orders?

- **What to Look For:** We want to see if expensive shipping pushes people to cancel. If orders with a "High" shipping cost have a much higher cancellation rate than "Low" shipping orders, we have solid proof that expensive freight hurts sales.

```
In [8]: print("--- 2.3 Analyzing Cancellation Rates by Freight Bracket: ---\n")

# Create an order-level copy to analyze freight brackets
orders_unique_freight = orders_unique.copy()

# 1. Create the quartile bins for freight ratio on the unique orders.
orders_unique_freight['freight_bracket'] = pd.qcut(orders_unique_freight['freight_ratio'], q=4, labels=[

# 2. Create a temporary binary column where 1 = canceled, 0 = everything else
orders_unique_freight['is_canceled'] = np.where(orders_unique_freight['order_status'] == 'canceled', 1, 0)

# 3. Group by the new freight bracket and calculate the mean of the 'is_canceled' column
cancellation_rates = orders_unique_freight.groupby('freight_bracket', observed=True)['is_canceled'].mean()

print(cancellation_rates)
print("\nInsight Check: Compare the 'High' bracket percentage against the 'Low' bracket.")
```

--- 2.3 Analyzing Cancellation Rates by Freight Bracket: ---

```
freight_bracket
Low          0.012440
Med-Low      0.004127
Med-High     0.000000
High         0.008294
Name: is_canceled, dtype: float64
```

Insight Check: Compare the 'High' bracket percentage against the 'Low' bracket.

Topic 3: Customer Behavior & Segmentation

Question 3.1: Payment Preferences

- **Stakeholder Question:** What are the most used payment methods, and do credit cards lead to higher spending?
- **Analyst Question:** What is the distribution of payment_type? Do customers using credit cards have a significantly higher AOV than those using other methods?
- **What to Look For:** Credit cards will likely be the most popular method. However, we want to see if customers actually spend more money per order when they use a credit card compared to cash-based methods (like boleto).

```
In [9]: print("\n--- 3.1 Payment Preferences ---")

# FIX: Group by payment type on unique orders to ensure average spend is accurate
payment_prefs = orders_unique.groupby('payment_type').agg(
    total_orders=('order_id', 'nunique'),
    average_spend=('total_payment_value', 'mean')
).sort_values(by='total_orders', ascending=False)

# Format the average spend for better readability
payment_prefs['average_spend'] = payment_prefs['average_spend'].round(2)
print(payment_prefs)
```

```
--- 3.1 Payment Preferences ---
              total_orders  average_spend
payment_type
credit_card             72104           165.85
boleto                 19187           144.36
voucher                 1498           105.14
debit_card              1483           140.27
credit_card, voucher     1097           145.71
voucher, credit_card     1083           151.95
credit_card, debit_card     1           152.82
```

Question 3.2: Installment Psychology

- **Stakeholder Question:** Do payment installments increase order value?
- **Analyst Question:** Does the availability of payment_installments correlate with customers buying more expensive items (total_item_value)?
- **What to Look For:** We are looking at the statistical correlation. If the number is positive (especially above 0.30), it proves that giving customers the option to pay in installments encourages them to buy more expensive products.

```
In [10]: print("--- 3.2 Installment Psychology ---\n")

installment_corr = orders_unique['payment_installments'].corr(orders_unique['total_payment_value'])

print(f"Correlation between installments and total spend: {installment_corr:.4f}")
if installment_corr > 0.5:
    print("Insight: Strong positive correlation. Customers definitely spend more when they can split paym")
elif installment_corr > 0.2:
    print("Insight: Moderate positive correlation. Installments likely encourage slightly higher spending")
else:
    print("Insight: Weak or no correlation. Splitting payments doesn't strongly drive up the total order")
```

--- 3.2 Installment Psychology ---

Correlation between installments and total spend: 0.3202

Insight: Moderate positive correlation. Installments likely encourage slightly higher spending.

Question 3.3: High-Value Customers (RFM Prep)

- **Stakeholder Question:** Which 10 cities have the highest-spending customers?
- **Analyst Question:** Which top 10 cities have the highest concentration of customers in the top 5% based on total payment value?
- **What to Look For:** We want to find out exactly where our VIP customers live. If the majority of the top spenders are located in just a few cities like São Paulo, the marketing team will know exactly where to focus their ad budget.

```
In [11]: print("--- 3.3 High-Value Customers (Top 5%) ---\n")

# 1. Calculate total spend per unique customer
customer_spend = orders_unique.groupby(['customer_unique_id', 'customer_city']).agg(
    total_spend=('total_payment_value', 'sum'),
    total_orders=('order_id', 'nunique')
).reset_index()

# 2. Find the 95th percentile threshold
top_5_threshold = customer_spend['total_spend'].quantile(0.95)

# 3. Filter for High-Value Customers (HVCs)
high_value_customers = customer_spend[customer_spend['total_spend'] >= top_5_threshold]

# 4. Count how many HVCs live in each city
city_hvc_counts = high_value_customers['customer_city'].value_counts().head(10).reset_index()
city_hvc_counts.columns = ['customer_city', 'hvc_count']

# Print formatted results
print(f"95th Percentile Spending Threshold: R$ {top_5_threshold:.2f}")
print(f"Total High-Value Customers Found: {len(high_value_customers)}\n")
print("Top 10 Cities with the Most High-Value Customers:")
print(city_hvc_counts)
```

--- 3.3 High-Value Customers (Top 5%) ---

95th Percentile Spending Threshold: R\$ 468.95
Total High-Value Customers Found: 4673

Top 10 Cities with the Most High-Value Customers:

	customer_city	hvc_count
0	sao paulo	598
1	rio de janeiro	342
2	belo horizonte	110
3	brasilia	107
4	curitiba	78
5	salvador	71
6	campinas	64
7	porto alegre	53
8	belem	42
9	guarulhos	40

Topic 4: Product Catalog & Satisfaction

Question 4.1: Volume vs. Value

- **Stakeholder Question:** Which product categories generate the most revenue and most sales?
- **Analyst Question:** Which product_category_name drives the highest total revenue, and which drives the highest volume of individual items sold?
- **What to Look For:** We want to separate the "cash cows" from the "high velocity" items. Expensive categories (like electronics) usually drive total revenue, while cheaper, everyday items (like home goods) drive the pure number of sales.

```
In [12]: print("--- 4.1 Volume vs. Value ---\n")

# 1. Aggregate revenue and volume by category
category_performance = df.groupby('product_category_name').agg(
    total_revenue=('price', 'sum'),
    total_items_sold=('order_item_id', 'count')
)

# 2. Apply English Translations for readability
translation_map = {'beleza_saude': 'Health & Beauty', 'relogios_presentes': 'Watches & Gifts', 'cama_mesas': 'Furniture'}
final_map = {k: f"{k} ({v})" for k, v in translation_map.items()}
# Map the index (category names), keeping the original if it's not in our dictionary
category_performance.index = category_performance.index.map(final_map)

# 3. Sort and print by Revenue
top_by_revenue = category_performance.sort_values(by='total_revenue', ascending=False).head(10)
print("Top 5 Categories by Revenue (Cash Cows):")
print(top_by_revenue, "\n")

# 4. Sort and print by Volume
top_by_volume = category_performance.sort_values(by='total_items_sold', ascending=False).head(10)
print("Top 5 Categories by Items Sold (High Velocity):")
print(top_by_volume)
```

--- 4.1 Volume vs. Value ---

Top 5 Categories by Revenue (Cash Cows):

product_category_name	total_revenue	total_items_sold
beleza_saude (Health & Beauty)	1233076.64	9464
relogios_presentes (Watches & Gifts)	1165898.98	5857
cama_mesa_banho (Bed, Table & Bath)	1022485.26	10945
esporte_lazer (Sports & Leisure)	954695.05	8431
informatica_acessorios (Computer Accessories)	888613.62	7643
moveis_decoracao (Furniture & Decor)	711927.69	8160
utilidades_domesticas (Housewares)	615628.69	6795
cool_stuff (Cool Stuff)	610204.10	3718
automotivo (Automotive)	578849.35	4139
brinquedos (Toys)	471197.48	4030

Top 5 Categories by Items Sold (High Velocity):

product_category_name	total_revenue	total_items_sold
cama_mesa_banho (Bed, Table & Bath)	1022485.26	10945
beleza_saude (Health & Beauty)	1233076.64	9464
esporte_lazer (Sports & Leisure)	954695.05	8431
moveis_decoracao (Furniture & Decor)	711927.69	8160
informatica_acessorios (Computer Accessories)	888613.62	7643
utilidades_domesticas (Housewares)	615628.69	6795
relogios_presentes (Watches & Gifts)	1165898.98	5857
NaN	309860.23	4430
NaN	470495.28	4268
automotivo (Automotive)	578849.35	4139

Question 4.2: The Late Penalty

- **Stakeholder Question:** How much do late deliveries lower customer review scores?
- **Analyst Question:** Is there a direct, quantifiable drop in the average review_score when an order's delivery_delay_days is greater than 0?
- **What to Look For:** We expect to see a huge drop in customer satisfaction when a package is late. We are looking for the exact difference in star ratings between on-time and late deliveries to show to the stakeholders.

```
In [13]: print("--- 4.2 The Late Penalty ---")

df_reviews_unique = df.dropna(subset=['review_score']).drop_duplicates(subset=['order_id']).copy()
late_penalty = df_reviews_unique.groupby('is_late_delivery')['review_score'].mean().reset_index()
late_penalty['delivery_status'] = late_penalty['is_late_delivery'].map({0: 'On Time / Early', 1: 'Late'})

print("\nAverage Review Score by Delivery Status:")
print(late_penalty[['delivery_status', 'review_score']].to_string(index=False))
```

--- 4.2 The Late Penalty ---

Average Review Score by Delivery Status:

delivery_status	review_score
On Time / Early	4.290458
Late	2.272136

This output shows the reality of the business: the raw average star rating drops significantly when a package is late.

```
In [14]: import scipy.stats as stats

# 1. SEPARATE THE DATA
# We create two distinct groups of review scores to compare against each other.
# .dropna() is crucial here to remove any blank scores that would break the math.
on_time_scores = df[df['is_late_delivery'] == 0]['review_score'].dropna() # Group A: On-time
late_scores = df[df['is_late_delivery'] == 1]['review_score'].dropna()    # Group B: Late

# 2. RUN THE STATISTICAL TEST
# Welch's T-Test compares the averages of our two groups.
# equal_var=False is a best practice here because we don't assume the variance
# (spread of the scores) is identical between on-time and late deliveries.
t_stat, p_value = stats.ttest_ind(on_time_scores, late_scores, equal_var=False)

# 3. OUTPUT THE RAW RESULTS
# The T-Statistic shows the size of the difference relative to the variation in the data.
print(f"T-Statistic: {t_stat:.2f}")
```

```
# The P-Value shows the probability that this drop in score happened purely by random chance.
print(f"P-Value: {p_value:.5e}")

# 4. INTERPRET THE BUSINESS IMPACT
# In data science, a P-Value under 0.05 (the 5% threshold) is the industry standard
# to definitively state that the result is statistically significant.
if p_value < 0.05:
    print("Insight: The drop in review scores due to late deliveries is statistically significant.")
```

T-Statistic: 102.28

P-Value: 0.00000e+00

Insight: The drop in review scores due to late deliveries is statistically significant.

This output provides the mathematical proof (using a T-Test) that the drop in reviews is directly caused by late deliveries and is not just a random coincidence in the data.

Question 4.3: Listing Quality

- **Stakeholder Question:** Do more photos and longer descriptions improve product review scores?
- **Analyst Question:** Do products with longer descriptions (product_description_lenght) or more photos (product_photos_qty) statistically receive higher review scores or lower return rates?
- **What to Look For:** We want to test a common business myth. If the correlation number is very close to 0, it gives us hard proof to tell stakeholders that spending hours writing long descriptions won't actually improve customer satisfaction.

```
In [15]: print("--- 4.3 Listing Quality ---")

# 1. Define the specific columns we want to test
listing_cols = ['product_description_lenght', 'product_photos_qty', 'review_score']

# 2. Calculate the correlation matrix using the deduplicated reviews dataset
listing_corr = df_reviews_unique[listing_cols].corr()

# 3. Extract the specific correlation values
desc_corr = listing_corr.loc['product_description_lenght', 'review_score']
photo_corr = listing_corr.loc['product_photos_qty', 'review_score']

# Print the mathematical results
print(f"Correlation: Description Length vs. Review Score = {desc_corr:.4f}")
print(f"Correlation: Photo Quantity vs. Review Score = {photo_corr:.4f}")

# 4. Provide the automated business insight
if abs(desc_corr) < 0.1 and abs(photo_corr) < 0.1:
    print("\nINSIGHT: The correlation is negligible (close to 0).")
    print("Takeaway for presentation: Longer descriptions and extra photos do not statistically guarantee")
else:
    print("\nINSIGHT: There is a notable correlation. Listing quality impacts satisfaction.")
```

--- 4.3 Listing Quality ---

Correlation: Description Length vs. Review Score = 0.0148

Correlation: Photo Quantity vs. Review Score = 0.0145

INSIGHT: The correlation is negligible (close to 0).

Takeaway for presentation: Longer descriptions and extra photos do not statistically guarantee better customer reviews.

Topic 5: Predictive Modeling (The "Late Penalty")

Question 5.1: Predicting Satisfaction

- **Stakeholder Question:** Can a Linear Regression model predict how much review scores drop per day of delay?
- **Analyst Question:** Can we mathematically predict the exact drop in customer satisfaction for every additional day a delivery is delayed past its estimated arrival date using a Linear Regression model?

Objective: We are moving past basic averages here. I will train a Machine Learning model (Linear Regression) to calculate exactly how much the star rating drops for every single day a package is late.

```
In [16]: print("--- Topic 5:- Advanced Analysis: Predictive Modeling ---\n")

from sklearn.linear_model import LinearRegression

# 1. Clean the dataset: Remove rows with missing data in key columns
ml_df = df.dropna(subset=['review_score', 'delivery_delay_days', 'freight_ratio']).copy()

# 2. Filter the data: Analyze only orders that were actually late
ml_df = ml_df[ml_df['delivery_delay_days'] > 0]

# 3. Define variables
X = ml_df[['delivery_delay_days', 'freight_ratio']]
y = ml_df['review_score']

# 4. Initialize and train the model
model = LinearRegression()
model.fit(X, y)

# 5. Output the results
print("--- Predictive Impact on Review Scores ---")
print(f"Base Score Intercept: {model.intercept_:.2f} stars")
print(f"For every 1 additional day of delay, the score drops by: {abs(model.coef_[0]):.3f} stars")

--- Topic 5:- Advanced Analysis: Predictive Modeling ---

--- Predictive Impact on Review Scores ---
Base Score Intercept: 2.49 stars
For every 1 additional day of delay, the score drops by: 0.015 stars
```



Animesh Sanghi | Data Analyst | MBA '28 (Analytics & Data Science) MUJ
9406570600 | animeshsanghi.da@gmail.com | [LinkedIn Profile](#) | [GitHub Profile](#)